

SheetAgent: Towards a Generalist Agent for Spreadsheet Reasoning and Manipulation via Large Language Models

Yibin Chen*
yibin_chen@tju.edu.cn
School of New Media and
Communication, Tianjin University
Tianjin, China

Yan Zheng†
yanzheng@tju.edu.cn
School of New Media and
Communication, Tianjin University
Tianjin, China

Jianye Hao
jianye.hao@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Yifu Yuan*
yuanyf@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Jinyi Liu
jyliu@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Hangyu Mao
hy.mao@pku.edu.cn
Independent Researcher
Beijing, China

Zeyu Zhang
3020208104@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Fei Ni
fei_ni@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Fuzheng Zhang
zhfzhkris@outlook.com
Independent Researcher
Beijing, China

Abstract

Spreadsheets are ubiquitous across the World Wide Web, playing a critical role in enhancing work efficiency across various domains. Large language model (LLM) has been recently attempted for automatic spreadsheet manipulation but has not yet been investigated in complicated and realistic tasks where reasoning challenges exist (e.g., long horizon manipulation with multi-step reasoning and ambiguous requirements). To bridge the gap with the real-world requirements, we introduce **SheetRM**, a benchmark featuring long-horizon and multi-category tasks with reasoning-dependent manipulation caused by real-life challenges. To mitigate the above challenges, we further propose **SheetAgent**, a novel autonomous agent that utilizes the power of LLMs. SheetAgent consists of three collaborative modules: *Planner*, *Informer*, and *Retriever*, achieving both advanced reasoning and accurate manipulation over spreadsheets without human interaction through iterative task reasoning and reflection. Extensive experiments demonstrate that SheetAgent delivers 20–40% pass rate improvements on multiple benchmarks over baselines, achieving enhanced precision in spreadsheet manipulation and demonstrating superior table reasoning abilities. More details and visualizations are available at

the project website. The datasets and source code are available at <https://anonymous.4open.science/r/SheetAgent>.

CCS Concepts

- **Computing methodologies** → **Natural language processing**;
- **Information systems** → **Decision support systems**.

Keywords

Agents, Large Language Models, Benchmark, Spreadsheet Reasoning and Manipulation

ACM Reference Format:

Yibin Chen, Yifu Yuan, Zeyu Zhang, Yan Zheng, Jinyi Liu, Fei Ni, Jianye Hao, Hangyu Mao, and Fuzheng Zhang. 2025. SheetAgent: Towards a Generalist Agent for Spreadsheet Reasoning and Manipulation via Large Language Models. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*, April 28–May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/3696410.3714962>

1 Introduction

Tabular data plays a crucial role in domains such as scientific research, finance, and marketing, where it is predominantly handled using spreadsheet systems. These systems, such as Google Sheets and Microsoft Excel, are useful for tasks including numerical calculations, data analysis, and visualisation [9, 13, 18]. However, processing these affairs often involves a significant amount of repetitive labor and consultation [6, 10]. Recent work [6, 19] has explored the automation of simple spreadsheet manipulation tasks. For example, *Highlight rows with sales volume greater than 40* in the “Orders” sheet shown in Figure 1. This task can be accomplished through simple queries and formatting. However, they fail to consider the more complex and realistic tasks that encompass more than simple first-order logic. For instance, *Highlight rows of database-related books with sales volume greater than 40*. The difficulty of this instruction

*Both authors contributed equally to this research.

†Corresponding author: Yan Zheng (yanzheng@tju.edu.cn)

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WWW '25, Sydney, NSW, Australia

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1274-6/25/04

<https://doi.org/10.1145/3696410.3714962>

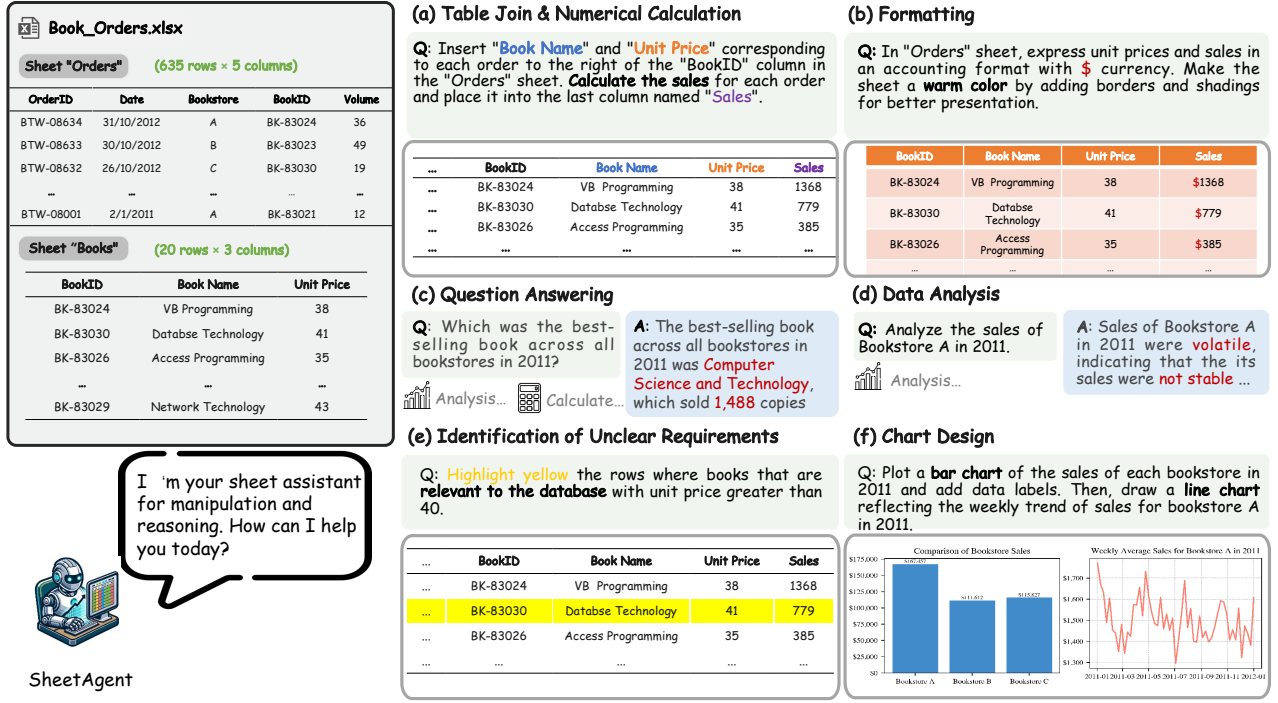


Figure 1: SheetAgent can handle diverse spreadsheet reasoning and manipulation tasks automatically. Given a large-scale spreadsheet with multiple sheets, SheetAgent shows its proficiency in visualization (f), achieves accurate manipulation on long horizon tasks (a, b) with consistent reasoning capabilities (c, d), even faced with the challenges like unclear requirements (e).

lies in identifying books related to the database, which cannot be achieved simply by understanding the semantics of column names, but rather by perceiving the specific content (what books are in the spreadsheet in this case) of the spreadsheet. Such scenarios are common because a complete spreadsheet¹ task may demand a multi-step reasoning process in conjunction with multiple sheets, and the user may not precisely define the required operations or ambiguously interpret the task instruction. Consequently, there is an urgent need for a new method to automate these tasks.

Designing such a method demands a combination of sophisticated sheet-based reasoning and manipulation capabilities. Previous work [12, 19, 27] has focused on precise spreadsheet manipulation while neglecting reasoning, limiting them to tasks with clear expressions and one-step reasoning. The emergence of large language models (LLMs) like GPTs [2, 29, 30] enables the integration of reasoning and manipulation capabilities. Extensive research [4, 16, 40] has shown that LLMs can reason over tables, handling tasks such as table question answering and fact verification. Given this context, we are motivated to explore the question: *Can we build a versatile agent adept at handling complex spreadsheet manipulation tasks with challenging reasoning factors using LLMs?* Designing an automated system is crucial [21, 24, 41, 45]. However, crafting such an automatic agent involves several challenges: (1) **Dynamic Changes of Spreadsheet:** Complex tasks often involve multiple operations,

resulting in dynamic spreadsheet content changes. Continuously feeding the entire spreadsheet into LLMs before each operation is impractical due to token limits and potential hallucination [7, 40]. (2) **Limited Table Understanding:** LLMs are predominantly trained in natural languages and show limited understanding of tables [20]. (3) **Lack of Benchmark:** There is an absence of a complicated benchmark demanding accurate reasoning and precise manipulation over spreadsheets. SheetCopilot [19] presents a benchmark for evaluating LLM performance in controlling spreadsheets. However, it simplifies real-world requirements, ignoring challenges like multi-step reasoning and long-horizon operations.

To address the dataset gap, we first introduce **SheetRM (Spreadsheet Reasoning and Manipulation Benchmark)**, a benchmark for developing and evaluating LLM-based agents for precise spreadsheet manipulation and advanced reasoning capabilities. Each task in SheetRM involves multiple subtasks that relies on reasoning abilities, derived from real-world Excel exam datasets. Moreover, it enables automatic evaluation with various metrics. We further present **SheetAgent**, a generalist agent for sheet manipulation and reasoning using LLMs. SheetAgent mainly consists of three components: the Planner, Informer, and Retriever. The Planner translates conceptual understandings into proficient code generation to manipulate spreadsheets. The Informer parses task demands and produces high-quality, task-specific SQL queries to understand the spreadsheet without needing to read the entire table, despite its dynamic changes. The Retriever retrieves instructive examples

¹A spreadsheet is a collection of sheets that are organized into a document. A table represents a structured arrangement of data in rows and columns. Each sheet within the spreadsheet contains a table.

to improve the robustness of solutions. We demonstrate that SheetAgent significantly outperforms other state-of-the-art baselines in diverse benchmarks. Our contributions are three-fold:

- We introduce SheetRM, a benchmark for developing and evaluating LLM-based agents to manipulate spreadsheets with advanced reasoning abilities. It includes more challenging tasks that reflect real-world requests and supports automatic evaluation with various metrics.
- We develop a versatile LLM-based agent SheetAgent, combining sheet manipulation and reasoning abilities to boost multifaceted interaction between humans and spreadsheets.
- Experimental results show that SheetAgent can be combined with any LLMs backbone and SheetAgent outperforms baselines across multiple benchmarks, achieving a 20–40% improvement in various metrics. These results highlight SheetAgent’s exceptional capabilities in spreadsheet manipulation and table reasoning.

2 SheetRM Benchmark

Unlike existing datasets [11, 19, 27] primarily designed for more precise spreadsheet manipulation, our goal is to construct a more realistic dataset, where tasks contain challenges such as complicated multi-step reasoning and vague requirements, to narrow the gap between simulation and real-world scenarios. To achieve this, we begin by sourcing spreadsheets from real-life Excel exam datasets. We collate a diverse set of spreadsheet operations commonly used in realistic scenarios and analyze the challenges faced when addressing spreadsheet tasks in practical settings. In brief, our SheetRM dataset is featured by the following elements, as outlined in Figure 2(a)-(d):

- **Multi category:** We summarize and collect 5 broad types and 36 subtypes of manipulation categories with corresponding 4 reasoning challenges. Each task includes an examination of both manipulation and reasoning abilities.
- **Reasoning-dependent manipulation:** Tasks include operations with multi-step reasoning over spreadsheets.
- **Long horizon:** Various subtasks constitute a complete task, which brings to agents the challenge of dynamic changes in spreadsheets.
- **Procedure evaluation:** We build an automated program evaluation approach for SheetRM that not only automates the determination of whether the full task is completed but also detects the completion of individual subtasks.

2.1 Task Schema

Each task is defined by the following three parts. See Figure 2 (upper right) for a visual demonstration.

Spreadsheet Assets. Each task consists of a spreadsheet as well as multiple sheets. We summarize the contents of the spreadsheet in a one-sentence natural language overview as context, as shown in Table 7, aiming to stimulate the internal knowledge of LLMs.

Task Instruction. A task instruction outlines the overall requirements expressed in natural language. Completing a task instruction requires a series of operations on the target spreadsheet.

Checklist. A task is paired with a checklist designed to evaluate its completion. Each item in the checklist corresponds to the evaluation of a fine-grained operation with tailored criteria. The automatic evaluation will be discussed in detail in Section 2.3.

Table 1: Basic statistics of SheetRM.

Item	Count
# Sheets	137
# Average Rows per File	300.82
# Average Columns per File	26.23
# Tasks	317
# Subtasks	1625

2.2 Dataset Construction

We gather and refine publicly available spreadsheets through a selection and cleaning process. Tasks are generated with both human and GPT-4 annotation. All the tasks are attached with verified answers, which enables model-free evaluation. The statistics of our curated dataset are shown in Table 1. Compared to the SheetCopilot benchmark, our SheetRM has a more granular and reasonable categorisation, holds more tasks with longer horizon and includes reasoning challenges. See Appendix A for a detailed comparison and more statistics.

Spreadsheet Files Collection. We began by gathering spreadsheets from a public examination question bank, carefully excluding files that were protected, corrupted, or inaccessible. To address privacy concerns, we modified sensitive information by adding noise to age data and anonymizing names, such as bookstores. Our selection aimed to cover multiple domains, ensuring diversity. Each file typically contains at least two sheets with a minimum of 20 rows and 5 columns. External dependencies were translated into natural language or embedded as feasible. Ultimately, we selected 41 spreadsheets with 137 sheets, averaging 300.82 rows and 26.23 columns per spreadsheet. This comprehensive collection provides a rich dataset for in-depth analysis. For additional details on our collection process, refer to Appendix A.1.

Task Generation. We begin by referring to websites about spreadsheet software skills and consult corporate staff about commonly used spreadsheet operations in their work. As shown in Figure 2, we conclude five coarse operation categories and their fine-grained specific operations for manipulation. Drawing insights from common table reasoning datasets like WikiTableQuestions, FeTaQA, and TabFact, we summarize four challenges in the process of sheet reasoning: (1) complex computation logic, (2) vague requirements, (3) incoherent data format and (4) information extraction. We detail these challenges in the Appendix A.4. Then, we instruct GPT-4 to propose *realistic* tasks that mimic user requests adhering to four guidelines: the tasks should only involve predefined operations, cover diverse manipulation categories, exhibit a long-horizon nature by encompassing multiple subtasks, and incorporate at least one subtask that presents the specified reasoning challenges. This process yields a compilation of 2316 subtasks. We eliminate semantically redundant entries for identical files to maintain uniqueness. To guarantee quality, our internal annotators manually validate subtasks using programming and specialized software such as Excel. Certain unreasonable subtasks are excluded throughout this process. By combining these subtasks for different spreadsheets considering horizon and complexity, we ultimately assemble 317 task instructions, encompassing a total of 1625 subtasks.

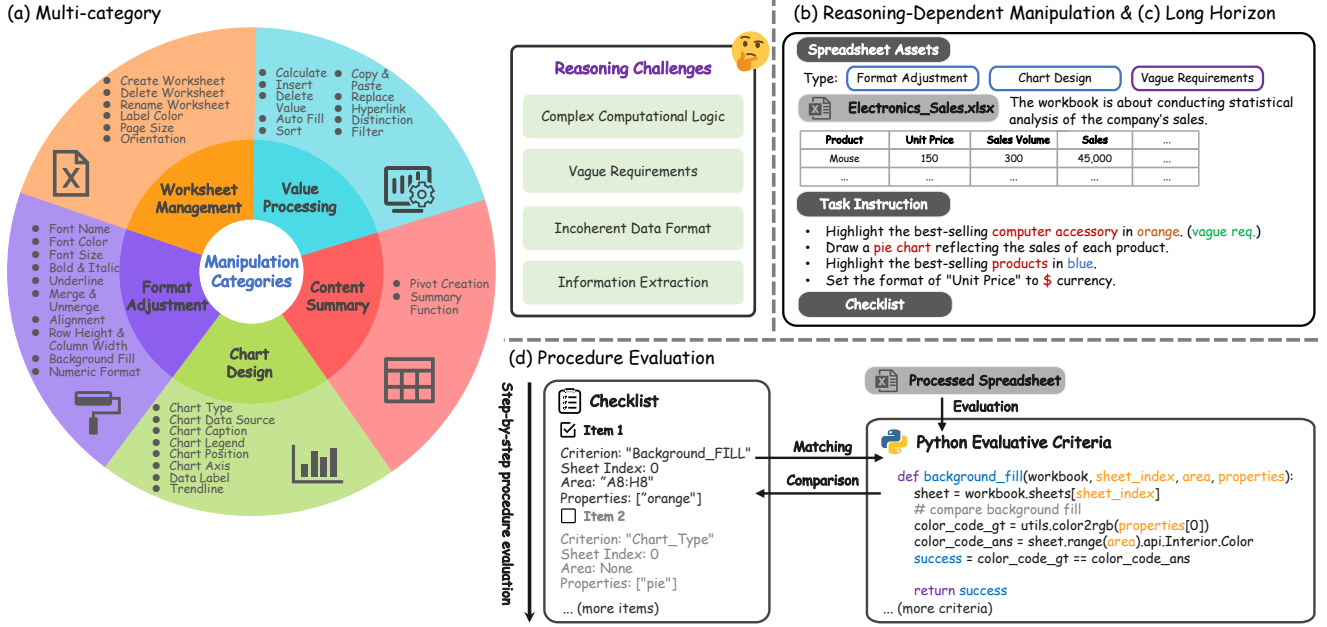


Figure 2: Overview and features of SheetRM. (a) **Multi-category:** SheetRM contain real-life tasks for multiple types of manipulation categories and reasoning challenges. Each task includes an examination of both manipulation and reasoning abilities. (b&c) **Long horizon and reasoning-dependent Manipulation:** An example task including three parts. Spreadsheet assets contain sheet data and one-sentence description with category of tasks. Then task instruction provides the requirements for the execution of the long horizon tasks. Checklist is designed for procedure evaluation. (d) **Procedure evaluation:** SheetRM automatically evaluates each task step-by-step via corresponding checklist and evaluative criterion to achieve procedure evaluation.

2.3 Automatic Evaluation

SheetCopilot [19] introduces a feasible method that determines task fulfillment by evaluating the alignment of key properties between the processed spreadsheet and ground truth candidates. However, this method fails to evaluate the accuracy of each operation involved, as a task may comprise multiple detailed intermediate steps. To address this challenge, we develop an automatic evaluation system that is model-free and tailored for each fine-grained operation. The advantage of that is that we are able to evaluate the performance of intermediate sub-task processes. A checklist is crafted for each task instruction. As illustrated in Figure 2, within the checklist, each evaluation item comprises a (Criterion, Sheet Index, Area, Properties) pair. We locate the comparison region in the target spreadsheet by (Sheet Index, Area). Then, corresponding Criterion is applied to verify whether the region aligns with the Properties. This design enables a detailed evaluation of LLMs' capabilities by assessing each step of task execution.

3 SheetAgent Framework

To quantify the challenges posed by SheetRM, we introduce an LLM-based agent framework SheetAgent. As outlined in Figure 3, SheetAgent consists of three key components: the Planner, the Informer, and the Retriever. The Planner generates Python code to manipulate the target spreadsheet, reasoning and acting to address complex tasks. The Informer supplies task-specific SQL queries, whose execution results provide crucial subviews of the spreadsheet,

narrowing the reasoning scope for the Planner and enhancing its ability to tackle complicated spreadsheet reasoning challenges. At each decision step, the Planner can perform more precise manipulations with subviews. When the Planner generates an incorrect solution, the Retriever is activated to fetch high-quality code examples from our curated repository, assisting the Planner in making more effective corrections.

3.1 Proficient Spreadsheet Manipulation with Planner

We design a Planner module to manipulate spreadsheets in SheetAgent. The way to interact with spreadsheets determines the precision of manipulation. Unlike SheetCopilot [19], which uses a set of language APIs, we adopt a code-centric approach to control spreadsheets. We find Python, compared with VBA, is more suitable for manipulating spreadsheets due to its alignment with the training corpora of most existing LLMs [3, 31]. This Python code-centric approach reduces the occurrence of hallucinations of LLMs.

Complex spreadsheet manipulation tasks often involve multiple steps. Achieving precise control over spreadsheets is challenging without an effective feedback mechanism. To address this, we devise a closed-loop planning process where the Planner interacts with the target spreadsheet, incorporating feedback and reflection. We first concatenate task instruction I , system prompt P^P , description D , and the initial sheet state s_0 (row and column count, headers, and data type of each column) as the input. Given

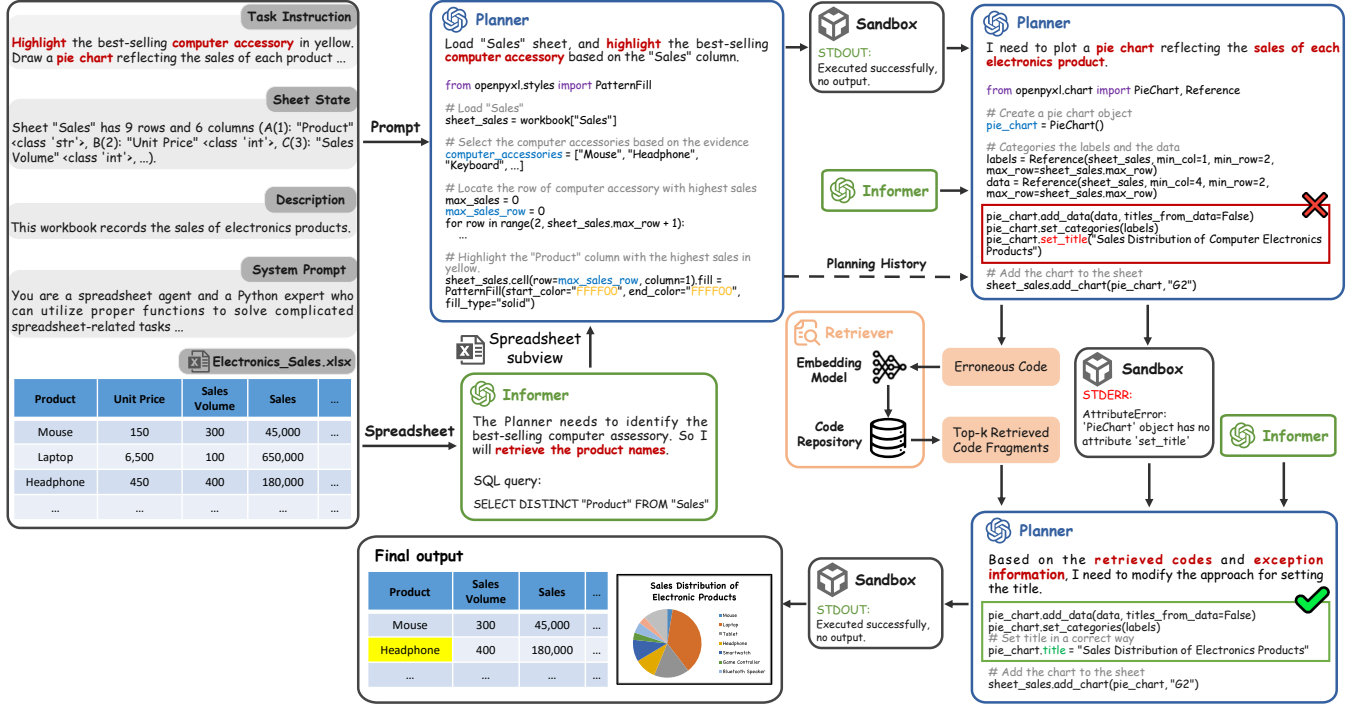


Figure 3: An illustration of SheetAgent. SheetAgent comprises three key components, including the Planner, the Informer, and the Retriever. The Planner interacts with the target spreadsheet via a virtual sandbox. The Informer provides subtask-specific SQLs, the execution results of which serve as the evidence for the Planner to handle reasoning challenges. The Retriever is invoked to retrieve similar tutorial code snippets upon encountering an error, effectively correcting the error.

a snapshot of the target spreadsheet at step t , the Planner generates action $a_t = \text{Planner}(a_t | I, P^t, D, s_t, h_{t-1})$, where h_{t-1} is the planning history. The action is evaluated in a sandbox with the feedback $o_t = \text{Sandbox}(a_t)$. If an error occurs, the Planner reflects and generates an adjusted action $a_t^* = \text{Planner}(a_t^* | I, P^t, D, s_t, h_{t-1}, o_t)$. Otherwise, the action is performed on the target spreadsheet. The spreadsheet is updated to a new state of s_{t+1} . The planning history is also updated to $h_t = (h_{t-1}, o_t, a_t)$. By this, Planner can achieve accurate manipulation with only the key information (i.e., the sheet state) of the spreadsheet rather than reading all the sheet data.

3.2 Accurate Spreadsheet Perception with Informer

Merely being aware of the sheet state does not equip the Planner to address the reasoning challenges shown in Figure 2. For instance, to fulfill the instruction illustrated in Figure 3, the Planner needs to discern which products qualify as computer accessories. However, the Planner struggles to query the spreadsheet effectively due to the absence of efficient mechanisms like SQLs and lacks the intrinsic ability to comprehend the data’s semantics. A feasible approach is constantly feeding the full spreadsheet into the Planner. However, considering the continuity of operations in a complicated task, a spreadsheet may experience multiple modifications, making it challenging to maintain a synchronized state within the Planner due to the token limit.

Therefore, we introduce the Informer to handle spreadsheets with arbitrary length and dynamic changes. Informer generates task-specific SQLs to perform queries. Initially, the tabular data in the target spreadsheet is extracted and stored in a lightweight database. At step t , the Informer’s objective is to select entries that align best with both the task instruction and the current step. To achieve this, we formulate the input of Informer by combining the system prompt P^t , the task instruction I , and previous actions $A_{t-1} = (a_1, \dots, a_{t-1})$ performed by the Planner. Then, Informer generates a SQL query $q_t = \text{Informer}(q_t | I, P^t, A_{t-1}, s_t)$. A_{t-1} is the reasoning trace of the Planner, enabling the Informer to generate more task-specific and robust SQLs. As shown in Figure 3, the execution result of the SQL query is a spreadsheet subview, which serves as evidence e_t for the Planner to reason over. This allows Planner to more accurately and efficiently perceive the target spreadsheet from key evidence, thereby addressing the reasoning challenges. Each time the Planner performs an operation, the spreadsheet in the database is updated to maintain synchronization.

3.3 Robust Solution Generation with Retriever

The Retriever advises the Planner during task planning, augmenting error corrections by sourcing relevant code from a code repository. We collect high-quality code from GitHub and craft tutorial examples for each manipulation category shown in Figure 2. We organize them into a compilation of code files. To improve efficiency, we

employ Milvus [35], an open-source vector database, as the code repository. To construct this repository, a sliding window technique is applied to traverse these files, extracting continuous lines of code within the window size. These code fragments C_{repo} are embedded into a set of vectors and stored in the code repository. The Retriever is invoked when the sandbox emits an error signal. We seek top- k similar code snippets C_{ret}^k as follows:

$$C_{ret}^k = \left\{ C_{repo}^i | C_{repo}^i \in C_{repo}, \forall C_{repo}^j \notin C_{ret}^k, \right. \\ \left. \text{sim}(\mathcal{E}(C_q), \mathcal{E}(C_{repo}^i)) > \text{sim}(\mathcal{E}(C_q), \mathcal{E}(C_{repo}^j)) \right\},$$

wherein $|C_{ret}^k| = k$, C_q refers to the erroneous code snippet, and $\text{sim}(\cdot)$ denotes cosine similarity. The embedding function $\mathcal{E}(\cdot)$ can be represented by any pretrained language model. Consequently, the top- k similar code snippets C_{ret}^k arranged in descending order are retrieved. These code snippets boost the replanning process of the Planner with $a_t^* = \text{Planner}(a_t^* | I, P^P, D, s_t, h_{t-1}, o_t, C_{ret}^k)$ for generating more robust and reliable solutions. We provide details of code collection in Appendix B.

4 Experiment

We conduct experiments on various tasks to answer the following research questions (RQs):

- **Versatility (RQ1):** Is SheetAgent adept at both spreadsheet manipulation and reasoning?
- **Universality (RQ2):** Can different LLMs benefit from the design of SheetAgent?
- **Difficulty (RQ3):** Why SheetRM is a challenging benchmark for existing methods?
- **Ablation (RQ4):** How do the modules within SheetAgent contribute to its overall effectiveness?

4.1 Experiment Setup

Dataset and Evaluation Metrics. We evaluate our approach SheetAgent on 5 diverse benchmarks. *SheetCopilot Benchmark* (SCB), a benchmark consisting of 221 tasks, is selected to mainly assess the manipulation ability. To measure the reasoning capability, we adopt three table reasoning tasks, including *WikiTableQuestions* (WTQ) [26], *FeTaQA* [23], and *TabFact* [5]. We report the performance on these tasks using their official evaluation pipeline. The 317 tasks in our *SheetRM* is used to comprehensively evaluate manipulation and reasoning capabilities. Refer to Appendix C for more details of these datasets. For manipulation tasks, we adopt Exec@1 and Pass@1 following SheetCopilot. Exec@1 measures the percentage of solutions without exceptions during execution. Pass@1 is used to evaluate the successful accomplishment of the task. In addition, we use the SubPass@1 to count the success rate of subtasks in each task to assess the instruction following capability. As for reasoning tasks, we chose distinct evaluation metrics. For WTQ and TabFact, accuracy is adopted as the evaluation metric. For FeTaQA, we report the sacreBLEU score [28].

Baselines. For SCB, we compare SheetAgent with SheetCopilot [19] and OS-Copilot [38], two LLM-based agent frameworks, and VBA [19], a method that generates and runs VBA code. For table-based reasoning tasks, we select fine-tuning based LLMs like TAPEX

[22] and OmniTab [17], and prompting-based LLMs such as DATER [40] and StructGPT [16]. To our knowledge, there is a lack of methods capable of comprehensive spreadsheet manipulation. Besides, various approaches [8, 11, 32] finetune LLMs for specific tasks like formatting and formula prediction but lack open source weights. Therefore, we mainly compare VBA, SheetCopilot, and OS-Copilot on SheetRM. We chose JSON as the table representation for its superior performance as shown in Table 5. See Appendix D for implementation details.

4.2 Versatility (RQ1)

To answer RQ1, we conduct various experiments on both spreadsheet manipulation and reasoning tasks. Table 2 shows the results for SCB. Using GPT-3.5 as the backbone, we observe that SheetAgent outperforms SheetCopilot with a remarkable **16.8** higher Pass@1. Even without the Informer and Retriever components, our method still surpasses others in both metrics. This indicates that the generated Python code is more robust and reliable compared to VBA code or language APIs. Following Li et al. [19], we use GPT-4 on a subset of SCB, including 20 tasks. Our SheetAgent also outperforms SheetCopilot and OS-Copilot with **15.0** and **10.0** higher Pass@1 respectively. These results demonstrate that SheetAgent can better leverage the power of LLMs to achieve more accurate spreadsheet manipulation.

Further experiments focus on assessing SheetAgent’s reasoning capability. We remove the Retriever as these tasks typically involve simpler operations like sorting and filtering. Results in Table 3 show that SheetAgent outperforms other baselines on WTQ and FeTaQA tasks, indicating its capability to provide precise and informative responses. Besides, SheetAgent surpasses all fine-tuning based methods, and achieves comparable performance as SOTA method UnifiedSKG [39] on TabFact. The results underscores the synergy between the Planner and Informer, which significantly enhances SheetAgent’s efficacy in table reasoning tasks. Full results are provided in Appendix E.1.

We compare SheetAgent with baselines on the SheetRM to evaluate reasoning and manipulation capabilities. For fair comparison, we have improved SheetCopilot based on the open-source version with error feedback functionality. Results in Table 2 show SheetAgent significantly outperforms other baselines in three aspects: (1) **Robust solution generation:** SheetAgent achieves an Exec@1 of **92.4** with GPT-3.5, indicating more robust solutions, meaning Python code generated by LLMs is more robust than VBA and language APIs. (2) **Strong manipulation proficiency:** SheetAgent is proficient in complex multi-category tasks, achieving a maximum SubPass@1 of **77.0**, more than double that of SheetCopilot with GPT-4. (3) **Advanced reasoning ability:** SheetAgent can solve more reasoning challenges, whereas SheetCopilot struggles significantly (Pass@1 **44.8 vs 2.2**). This reflects the superior reasoning capabilities of SheetAgent. Additionally, we evaluate our SheetAgent using the latest GPT-4o and GPT-o1. As the model capabilities increase, it is observed that SheetAgent achieves better performance on the SheetRM benchmark, with Pass@1 scores of **46.1** and **62.5**, respectively. We also provide an illustrative case in Appendix F to further demonstrate why SheetAgent outperforms SheetCopilot in tasks with reasoning challenges.

Table 2: Performance comparison of different methods on SCB and our SheetRM. * denotes results on a subset of SCB with 20 representative tasks. Best results are bolded and second-best results are underlined.

Method	SCB		SheetRM		
	Exec@1 ↑	Pass@1 ↑	Exec@1 ↑	Pass@1 ↑	SubPass@1 ↑
VBA (GPT-3.5)	77.8	37.1	56.2	2.8	14.5
OS-Copilot (GPT-4)	/	<u>60.0</u> *	74.8	20.5	56.6
SheetCopilot (GPT-3.5)	87.3	44.3	68.1	0	16.2
SheetCopilot (GPT-4)	65.0*	55.0*	52.7	2.2	30.7
SheetAgent (GPT-3.5)	94.1	61.1	92.4	31.2	69.8
- w/o Informer+Retriever	<u>88.7</u>	<u>50.7</u>	88.6	11.4	57.5
SheetAgent (GPT-4)	90.0 *	70.0 *	89.3	44.8	77.0
SheetAgent (GPT-4o)	/	/	<u>93.1</u>	<u>46.1</u>	<u>79.8</u>
SheetAgent (GPT-o1)	/	/	96.2	62.5	87.5

Table 3: Results of different methods on three table reasoning benchmarks. Best results are bolded and second-best results are underlined.

Method	WTQ	TabFact	FeTaQA
Fine-tuning based LLMs			
TAPEX [22]	57.5	84.2	34.7
UnifiedSKG [39]	49.3	85.4	33.4
OmniTab [17]	<u>62.8</u>	82.8	<u>34.9</u>
Prompting based LLMs			
GPT-3 CoT [4]	45.7	76.0	27.0
Binder [7]	59.9	82.9	31.6
DATER [40]	61.6	80.7	30.9
StructGPT [16]	52.2	81.2	32.5
SheetAgent (GPT-3.5)	64.4	<u>84.8</u>	36.7

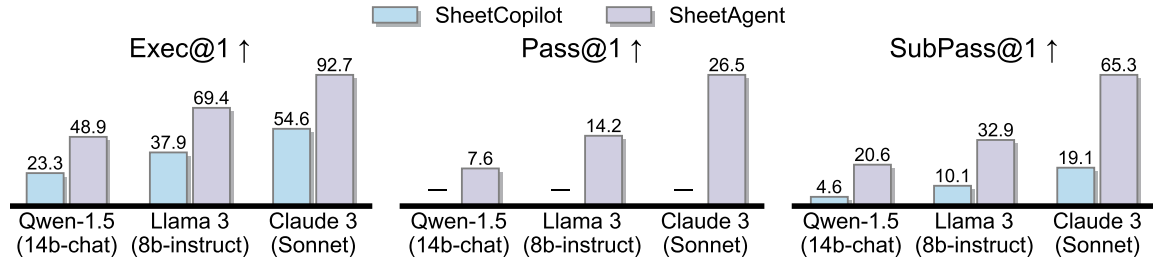


Figure 4: Performance on SheetRM for other LLM backbones. “—” means Pass@1=0. These backbones benefit significantly from the design of SheetAgent compared to SheetCopilot.

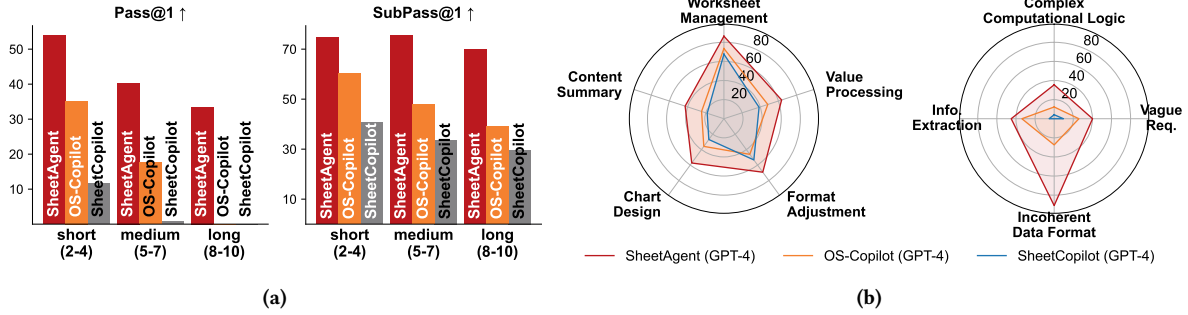


Figure 5: Comparison between SheetAgent and SheetCopilot with GPT-4. (a) Comparison of Pass@1 and SubPass@1 under different task horizon levels. (b) Pass rate of different manipulation categories (left) and reasoning challenges (right).

4.3 Universality (RQ2)

To answer RQ2, we compare our SheetAgent with SheetCopilot across various LLM backbones on SheetRM. As presented in Table 2 and Figure 4, SheetAgent shows remarkable improvements in all evaluated metrics on diverse backbones such as GPTs and Claude. Despite with smaller, open-source backbones, we can continue to observe the same results. Furthermore, SheetCopilot fails to pass any task completely on open-source models, and possesses lower Exec@1 scores, highlighting its challenges in generating feasible solutions. These results confirm the universality of SheetAgent, illustrating its consistent performance improvements across different LLM backbones regardless of scale. Meanwhile, we note that the

differences between various LLM backbones mainly stem from their fundamental capabilities, such as instruction following and code generation abilities. We further analyze failure cases on SheetRM in Appendix G to identify the deficiencies and opportunities for improvement in SheetAgent under different LLM backbones.

4.4 Difficulty (RQ3)

We explore the challenges of our proposed SheetRM benchmark from three perspectives: task horizon, task categories, and reasoning challenges. We compare SheetAgent with SheetCopilot and OS-Copilot against the same GPT-4 backbone. Tasks are categorized into three levels based on their horizon: short (2-4), medium

Table 4: Ablation study of different proposed components in SheetAgent on SheetRM dataset.

Method	Exec@1 ↑	Pass@1 ↑	SubPass@1 ↑
SheetAgent (GPT-3.5)	92.4	31.2	69.8
-w/o Informer	95.3 (+2.9)	13.2 (-18.0)	65.8 (-4.0)
-w/o Retriever	84.2 (-8.2)	19.9 (-11.3)	63.7 (-6.1)
-w/o Informer+Retriever	88.6 (-3.8)	11.4 (-19.8)	57.5 (-12.3)

Table 5: Ablation study on different representations. Best results are bolded and suboptimal results are underlined.

Representation	WTQ	SheetRM		
		Exec@1 ↑	Pass@1 ↑	SubPass@1 ↑
JSON	63.3	92.4	31.2	69.8
DFLoader	59.7	90.9	30.0	67.5
Markdown	58.6	90.2	29.0	65.4
HTML	<u>62.1</u>	85.2	23.3	57.5

(5-7), and long (8-10). As depicted in Figure 5(a), both methods exhibit a decreasing trend in Pass@1 and SubPass@1 as task horizon increases, indicating the difficulty of long-horizon tasks in our benchmark. Furthermore, Figure 5(b) presents the performance across different manipulation categories and reasoning challenges by evaluating subtask success rates. Both methods struggle with more complex tasks like chart design and content summary. Additionally, SheetCopilot can hardly address reasoning challenges. These findings highlight the challenges SheetRM poses in domains requiring consistent, robust reasoning and manipulation.

4.5 Ablation (RQ4)

Effects of Each Module. Table 4 reveals the effects of SheetAgent modules. Pass@1 drops dramatically without Informer, indicating its vital role in handling reasoning challenges by providing relevant information. Exec@1 also decreases sharply without Retriever, showing that high-quality examples help the Planner generate reliable code. Without both Informer and Retriever, SheetAgent performs poorest, highlighting the need for both reasoning and manipulation capabilities to tackle complex tasks effectively. Combining the results in Table 2, even with only the Planner, SheetAgent performs decently compared to SheetCopilot, showcasing the benefits of a code-centric approach.

Table Representations. Tabular data requires reliable representations for LLMs to recognize attribute relationships. We ablate four table representations—JSON, DFLoader, Markdown, and HTML—for SheetAgent on WTQ and SheetRM. Results in Table 5 show JSON outperforms other formats. HTML performs poorly on SheetRM due to verbosity and token limits. We provide illustration of different representations in Appendix E.3 with in-depth analysis. More additional ablations about temperature and vision-enabled SheetAgent can be found in Appendix E.

5 Related Work

LLMs for Table Reasoning. Recent research [36, 37] has demonstrated the excellent ability of LLMs for table reasoning tasks. Chen

[4] showcased that LLMs like GPT-3 [2] are capable of reasoning over tables. Binder [7] leverages Codex [3] to generate executable SQL programs to answer table-based questions. DATER [40] decomposes the table and question into finer granularity descriptions through Codex. StructGPT [16] designs an LLM-based framework for structured data and uses it for table question answering. However, these methods are tailored for tasks like question answering or fact verification, typically involving direct queries or explicit statements. As a result, they struggle to handle long-horizon manipulation tasks because of dynamic changes and token limits.

Automatic Spreadsheet Manipulation. Early research [1, 10–12] focus on leveraging program synthesis to guide spreadsheet manipulation. However, these methods fail to generate effective programs without high-quality query specifications. To address this, some work [6, 8, 14, 32] employ deep learning methods to automate spreadsheet manipulation tasks. Despite excellent performance in narrow domains like formatting and formula prediction, they cannot handle a broader range of operations. Given the remarkable performance of LLMs on various tasks [33, 34, 43], their use for comprehensive spreadsheet manipulation has been explored [19, 27, 38, 42, 44]. Payan et al. [27] utilizes LLMs to generate OfficeScripts code with multiple domains. SheetCopilot [19] builds an autonomous agent for invoking custom APIs to manipulate spreadsheets. TableGPT [42] fine-tunes an LLM to understand and operate on tables using external functions. OS-Copilot [38] proposes an OS-oriented agent framework that automates spreadsheet manipulation. However, both approaches simplify real-world requirements and overlook key reasoning challenges such as unclear expressions and multi-step logic. Unlike them, we further explore these real-life reasoning challenges and propose a collaborative agent framework that explicitly tackles spreadsheet tasks requiring deeper reasoning.

6 Conclusion

In this work, we introduce SheetRM, a more complex and realistic benchmark designed to evaluate the capabilities of agents in performing precise spreadsheet manipulations that require advanced reasoning abilities. Furthermore, We introduce SheetAgent that leverages the power of LLMs to tackle these challenging tasks. Comprehensive experiments have been conducted to assess the reasoning and manipulation proficiency of SheetAgent. We anticipate that SheetRM will serve as a cornerstone for advancing the development of sophisticated generalist agents dedicated to spreadsheet tasks. Furthermore, we hope SheetAgent can alleviate the burden of tedious sheet transactions through automated workflows. While SheetAgent demonstrates strong performance, we acknowledge several limitations like library coverage and token usage, detailed in Appendix I, which we leave as future work. We also hope to push the boundaries of automatic spreadsheet processing.

Acknowledgments

This work is supported by the National Key Research and Development Program of China (Grant No. 2024YFE0210900), the National Natural Science Foundation of China (Grant Nos. 62106172, 62422605, 92370132), the National Key Research and Development Program of China (Grant No. 2022ZD0116402) and the Xiaomi Young Talents Program of Xiaomi Foundation.

References

- [1] Matej Baloĝ, Alexander L Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. 2016. DeepCoder: Learning to Write Programs. In *International Conference on Learning Representations*.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [4] Wenhu Chen. 2023. Large Language Models are few (1)-shot Table Reasoners. In *Findings of the Association for Computational Linguistics: EACL 2023*. 1090–1100.
- [5] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2019. TabFact: A Large-scale Dataset for Table-based Fact Verification. In *International Conference on Learning Representations*.
- [6] Xinyun Chen, Petros Maniatis, Rishabh Singh, Charles Sutton, Hanjun Dai, Max Lin, and Denny Zhou. 2021. Spreadsheetcoder: Formula prediction from semi-structured context. In *International Conference on Machine Learning*. PMLR, 1661–1672.
- [7] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, et al. 2022. Binding Language Models in Symbolic Languages. In *The Eleventh International Conference on Learning Representations*.
- [8] Haoyu Dong, Jiong Yang, Shi Han, and Dongmei Zhang. 2020. Learning Formatting Style Transfer and Structure Extraction for Spreadsheet Tables with a Hybrid Neural Network Architecture. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2389–2396.
- [9] Alexander Edeling, Shuba Srinivasan, and Dominique M Hanssens. 2021. The marketing-finance interface: A new integrative review of metrics, methods, and findings and an agenda for future research. *International Journal of Research in Marketing* 38, 4 (2021), 857–876.
- [10] Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices* 46, 1 (2011), 317–330.
- [11] Sumit Gulwani, William R Harris, and Rishabh Singh. 2012. Spreadsheet data manipulation using examples. *Commun. ACM* 55, 8 (2012), 97–105.
- [12] Sumit Gulwani and Mark Marron. 2014. Nlyze: Interactive programming by natural language for spreadsheet data analysis and manipulation. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 803–814.
- [13] Md Morshadul Hasan, József Popp, and Judit Oláh. 2020. Current landscape and influence of big data on finance. *Journal of Big Data* 7, 1 (2020), 1–17.
- [14] Wanrong He, Haoyu Dong, Yihui Gao, Zhichao Fan, Xingzhuo Guo, Zhitao Hou, Xiao Lv, Ran Jia, Shi Han, and Dongmei Zhang. 2023. HermEs: Interactive Spreadsheet Formula Prediction via Hierarchical Formulæ Expansion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8356–8372.
- [15] Jonathan Herzig, Paweł Krzysztof Nowak, Thomas Mueller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly Supervised Table Parsing via Pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 4320–4333.
- [16] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. 2023. StructGPT: A General Framework for Large Language Model to Reason over Structured Data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9237–9251.
- [17] Zhengbao Jiang, Yi Mao, Pengcheng He, Graham Neubig, and Weizhu Chen. 2022. OmniTab: Pretraining with Natural and Synthetic Data for Few-shot Table-based Question Answering. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 932–942.
- [18] Sean Kandel, Andreas Paepcke, Joseph M Hellerstein, and Jeffrey Heer. 2012. Enterprise data analysis and visualization: An interview study. *IEEE transactions on visualization and computer graphics* 18, 12 (2012), 2917–2926.
- [19] Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and ZHAO-XIANG ZHANG. 2024. SheetCopilot: Bringing software productivity to the next level through large language models. *Advances in Neural Information Processing Systems* 36 (2024).
- [20] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Faiman, Dongmei Zhang, and Surajit Chaudhuri. 2023. Tablegpt: Table-tuned gpt for diverse table tasks. *arXiv preprint arXiv:2310.09263* (2023).
- [21] Jinyi Liu, Yifu Yuan, Jianye Hao, Fei Ni, Lingzhi Fu, Yibin Chen, and Yan Zheng. 2024. Enhancing Robotic Manipulation with AI Feedback from Multimodal Large Language Models. *arXiv preprint arXiv:2402.14245* (2024).
- [22] Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. 2021. TAPEX: Table Pre-training via Learning a Neural SQL Executor. In *International Conference on Learning Representations*.
- [23] Linyong Nan, Chiachun Hsieh, Ziming Mao, Xi Victoria Lin, Neha Verma, Rui Zhang, Wojciech Kryściński, Hailey Schoolkopf, Riley Kong, Xiangru Tang, et al. 2022. FeTaQA: Free-form Table Question Answering. *Transactions of the Association for Computational Linguistics* 10 (2022), 35–49.
- [24] Fei Ni, Jianye Hao, Shiguang Wu, Longxin Kou, Jiahsun Liu, Yan Zheng, Bin Wang, and Yuzheng Zhuang. 2024. Generate Subgoal Images before Act: Unlocking the Chain-of-Thought Reasoning in Diffusion Model for Robot Manipulation with Multimodal Prompts. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13991–14000.
- [25] Vaishali Pal, Evangelos Kanoulas, and Maarten Rijke. 2022. Parameter-Efficient Abstractive Question Answering over Tables or Text. In *Proceedings of the Second DialDoc Workshop on Document-grounded Dialogue and Conversational Question Answering*. 41–53.
- [26] Panupong Pasupat and Percy Liang. 2015. Compositional Semantic Parsing on Semi-Structured Tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 1470–1480.
- [27] Justin Payan, Swaroop Mishra, Mukul Singh, Carina Negreanu, Christian Poelitz, Chitta Baral, Subhro Roy, Rasika Chakravarthy, Benjamin Van Durme, and Elnaz Nouri. 2023. InstructExcel: A Benchmark for Natural Language Instruction in Excel. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 4026–4043.
- [28] Matt Post. 2018. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*. 186–191.
- [29] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [30] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. (2019).
- [31] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [32] Mukul Singh, José Cambronero, Sumit Gulwani, Vu Le, Carina Negreanu, Elnaz Nouri, Mohammad Raza, and Gust Verbruggen. 2023. Format5: Abstention and Examples for Conditional Table Formatting with Natural Language. *Proceedings of the VLDB Endowment* 17, 3 (2023), 497–510.
- [33] Hongda Sun, Yuxuan Liu, Chengwei Wu, Haiyu Yan, Cheng Tai, Xin Gao, Shuo Shang, and Rui Yan. 2024. Harnessing Multi-Role Capabilities of Large Language Models for Open-Domain Question Answering. In *Proceedings of the ACM on Web Conference 2024*. 4372–4382.
- [34] Bo Wang, Jing Ma, Hongzhan Lin, Zhiwei Yang, Ruichao Yang, Yuan Tian, and Yi Chang. 2024. Explainable Fake News Detection With Large Language Model via Defense Among Competing Wisdom. In *Proceedings of the ACM on Web Conference 2024*. 2452–2463.
- [35] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.
- [36] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*.
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [38] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [39] Tianbao Xie, Chen Henry Wu, Peng Shi, Ruiqi Zhong, Torsten Scholak, Michihiro Yasunaga, Chien-Sheng Wu, Ming Zhong, Pengcheng Yin, Sida I Wang, et al. 2022. UnifiedSKG: Unifying and Multi-Tasking Structured Knowledge Grounding with Text-to-Text Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 602–631.
- [40] Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. 2023. Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 174–184.
- [41] Yifu Yuan, HAO Jianye, Yi Ma, Zibin Dong, Hebin Liang, Jinyi Liu, Zhixin Feng, Kai Zhao, and YAN ZHENG. 2024. Uni-RLHF: Universal Platform and Benchmark Suite for Reinforcement Learning with Diverse Human Feedback. In *The Twelfth International Conference on Learning Representations*.
- [42] Liangyu Zha, Junlin Zhou, Liyao Li, Rui Wang, Qingyi Huang, Saisai Yang, Jing Yuan, Changbao Su, Xiang Li, Aofeng Su, et al. 2023. Tablegpt: Towards unifying tables, nature language and commands into one gpt. *arXiv preprint arXiv:2307.08674* (2023).

- [43] Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2024. Agentcf: Collaborative learning with autonomous language agents for recommender systems. In *Proceedings of the ACM on Web Conference 2024*. 3679–3689.
- [44] Wenqi Zhang, Yongliang Shen, Weiming Lu, and Yueting Zhuang. 2024. Data-Copilot: Bridging Billions of Data and Humans with Autonomous Workflow. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [45] Yan Zheng, Xiaofei Xie, Ting Su, Lei Ma, Jianye Hao, Zhaopeng Meng, Yang Liu, Ruimin Shen, Yingfeng Chen, and Changjie Fan. 2019. Wuji: Automatic online combat game testing using evolutionary deep reinforcement learning. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 772–784.

A Details of SheetRM Benchmark

A.1 Details of Dataset Collection

Spreadsheet Collection. The spreadsheets curated in SheetRM are derived from an online examination question bank. We filter out files that are protected, corrupted, or otherwise inaccessible. Within each spreadsheet, the first row of each column must include a header, with the actual data entries starting from the second row. Besides, we ensure all data in each sheet begin from cell A1. We assume that these spreadsheets have already undergone a process to remove some personal information. However, to minimize privacy risk by leaking important personal information, we further implement measures to ensure that no privacy issues arise. Specifically, we modify potentially sensitive information, such as adding noise to the age data and anonymizing bookstore names to general labels like Bookstore A, Bookstore B, etc.

Task Verification. As mentioned in Section 2.2, we instruct GPT-4 to generate realistic tasks that mimic user requests adhering to four guidelines: the tasks should only involve predefined operations, cover diverse manipulation categories, exhibit a long-horizon nature by encompassing multiple subtasks, and incorporate at least one subtask that presents the specified reasoning challenges. This procedure yields a collection of 2316 subtasks. We use GPT-3.5 to filter task instructions that have a lot of semantic duplication to maintain uniqueness. After this, 1973 subtasks are reserved. Furthermore, our internal annotators verify these subtasks manually to ensure quality, which increases the probability that they will be completed by LLMs. Specifically, we adopt two strategies: (1) programming and (2) specialized software. For programming, we ask our internal annotators to write code to complete specific subtask. For specialized software, we use Microsoft Excel to solve the subtask. We accept the subtask only if both strategies solve the subtask. This cross-way validation approach guarantees the reliability of the subtasks. We obtain 1625 subtasks after this process. Finally, we combine these subtasks for different spreadsheets considering horizon and complexity, which leads to 317 task instructions.

A.2 Comparison Between SheetRM and SCB

We conclude the differences that highlight the advantages of our proposed SheetRM dataset compared with SCB as follows:

- **More sheets:** The number of spreadsheet files in SheetRM and SCB is comparable. Besides, SheetRM maintains more spreadsheet files and sheets than SCB (**41 vs 28 & 137 vs 31**). Each spreadsheet file contains more complex logical relationships and information.
- **More subtasks and longer task horizon:** As shown in Table 6, SheetRM maintains more subtasks (**1625 vs 431**) with longer

horizon tasks (**averaging 5.13 vs 1.95**). Detailed task length distribution is presented in Figure 7.

- **Broader categories and more reasonable division:** SCB categorizes tasks into 6 main types: Entry & Manipulation, Formatting, Pivot Tables, Charts, Formulas, and Management, which results in unbalanced coverage and vague definitions. For example, Formula is basically a type of numerical computation and overlaps with Management and Manipulation, etc. In contrast, SheetRM divides the **5 major categories and 36 sub-categories** from coarse to fine and minimizes the overlap of sub-operations. We believe this allows for a better evaluation of the agents.
- **Finer-grained and more flexible evaluation:** We propose an **automated checklist-based evaluation** in SheetRM that is flexible and accurate for each subtask in the middle of a process, whereas SCB directly compares the final spreadsheets with the ground truth spreadsheets, ignoring the intermediate process of evaluation.
- **Introduction of reasoning challenges:** It is worth noting that reasoning challenges are innovatively introduced combined with manipulation in SheetRM. In real-world spreadsheet tasks, it is often necessary to reason and analyze problems and data in order to carry out operations. The SCB simplifies the task objectives by assessing only the LLM’s ability to manipulate spreadsheets. Instead, our proposed SheetRM presents more realistic and challenging tasks. Please refer to Appendix A.4 for further elaboration.

A.3 Detailed Statistics of Dataset

Spreadsheet Files. We provide more detailed statistics of our SheetRM dataset. We collect spreadsheets covering multiple fields. As illustrated in Figure 6 (Left), we categorize these spreadsheet files into five main fields, reflecting the significant areas where spreadsheets are frequently employed to handle a variety of tasks. We manually annotate a short natural language description as a summary for each spreadsheet file, aiming to stimulate inherent knowledge of LLMs. Each description provides an overview for LLMs to better understand the background information. We provide the descriptions in Table 7.

Task Instruction. We cluster the commonly used operation when working with spreadsheets into five categories, namely **Value Processing, Worksheet Management, Format Adjustment, Chart Design, and Content Summary**. For each manipulation category, we further break it down into fine-grained operations. We believe these operations can cover most spreadsheet affairs. The description of these operations is introduced in Table 8. Figure 6 (Right) demonstrates the distribution of verb-noun phrases within our 317 task instructions. We highlight the ten most frequent root verbs and their four primary associated nouns, showcasing the diversity of task instructions in the SheetRM dataset. Additionally, we show the distributions of the number of manipulation categories and subtasks for these task instructions (see Figure 7 (Left)). The majority of tasks span 2 or 3 manipulation categories, with a decent portion encompassing 4 categories, underscoring the diversity of tasks in the SheetRM dataset. We further count the number of subtasks in each task. As displayed in Figure 7, each task includes at least 2 sub-tasks, with the most complex extending to 10. Predominantly,

Table 6: Comparison of statistical data between SheetRM and SCB.

Dataset Name	# Files	# Sheets	# Task Instructions	# Subtasks	Avg. of Task Length	Median of Task Length	Max Task Length
SheetRM (Ours)	41	137	317	1625	5.13	5	10
SCB	28	31	221	431	1.95	2	7

the tasks vary in length from 3 to 7. This reflects the long horizon feature of SheetRM, which poses a significant challenge to LLMs.

A.4 Explanation of Reasoning Challenges

Our SheetRM dataset stands out from other spreadsheet manipulation collections due to its emphasis on reasoning-dependent manipulation. Specifically, each task incorporates reasoning challenges. We draw inspiration from several popular table reasoning tasks, including table question answering datasets WikiTableQuestions and FeTaQA, and table fact verification task TabFact. We analyze cases within these datasets that most models struggled with and identify four types of reasoning challenges, namely **Complex Computational Logic**, **Vague Requirements**, **Incoherent Data Format**, and **Information Extraction**. We find that these reasoning challenges are prevalent in real-world spreadsheet manipulation tasks due to the diversity of human expression. Thus, integrating practical insights, we incorporate these reasoning challenges into our spreadsheet manipulation tasks. We elaborate these challenges with descriptions and specific examples:

Complex Computational Logic

Description:

Problems that require more than one reasoning steps to be solved.

Example Sheet:

Name	Date of Entry	Educational Qualification	Salary
Alice	3/1/2001	Master	11,100
Bob	12/1/2006	Bachelor	10,350
...	...	...	...
John	1/9/2011	Doctor	41,100

Instruction:

Which period, 2001-2006 or 2007-2012, had a higher proportion of employees with bachelor's degrees? For the period with the higher proportion, calculate the average salary of the undergraduate employees and put it in cell E1.

Challenge:

To fulfill this instruction, the capability of multi-step reasoning is required.

Vague Requirements

Description:

Problems that refer to incomplete or ambiguous specifications which lack clarity and precision, making it challenging to understand and fulfill the intended goals or objectives.

Example Sheet:

BookID	Book Name	Unit Price
BK-83024	VB Programming	38
BK-83026	Access Programming	35
...	...	...
BK-83029	Network Technology	43

Instruction:

Highlight database-related books in yellow.

Challenge:

To fulfill this instruction, Reasoning over the sheet contents to identify which books are relevant to the database.

Incoherent Data Format

Description:

Problems that arise when the description provided pertains to the spreadsheet data, yet the units or formats mentioned do not align with those represented in the spreadsheet.

Example Sheet:

Name	Date of Birth
Alice	12/27/1964
Bob	9/28/1974
...	...
John	7/19/1987

Instruction:

Mark the names of employees born after 1985-1-1 in red.

Challenge:

To fulfill this instruction, the "Date of Birth" column should be inferred to align the format.

Information Extraction

Description:

Problems that require specific information to be extracted from the spreadsheet.

Example Sheet:

Venue	Opponent	Final Score
Memphis, Tennessee, USA	Jim Courier	7-5, 6-7, 6-7
Australian Open, Melbourne, Australia	Pete Sampras	6-7, 4-6, 4-6
...	...	...
Estoril, Portugal	Albert Costa	6-2, 3-6

Instruction:

Extract the scores from the first round of the finals into the new column "First Round Score".

Challenge:

To fulfill this instruction, Information about the "Final Score" is required to determine how to extract the first round score.

B Details of Code Collection for the Retriever

The Retriever’s code comes from GitHub open-source projects and external Python libraries like *openpyxl* and *pandas*, focusing on high-quality, popular code to ensure data representativeness. We organized these codes by operations covered in SheetRM and then abstracted them for universality. For operations without existing code, we gathered more from the same sources or wrote code ourselves, ensuring coverage of all defined operations. The organized code is related to the corresponding tasks, mainly demonstrating the application programming interfaces and providing high-level guidance. However, the specific implementations of these APIs and the generated solutions are different. Since we abstracted and encapsulated the collected codes, we only provided information on how to operate in it, while the application of the actual data is relevant to the task scenario. Thus, we anticipate that LLMs learn from the knowledge provided by these code snippets and reflect on past trajectories to generate more robust and higher-quality solutions to the task.

C Dataset Details

The details of datasets mentioned in Section 4.1 are provided as follows:

- **WikiTableQuestions** includes intricate questions created by crowd workers from Wikipedia tables. These questions necessitate multiple advanced operations like comparison, aggregation, and arithmetic, demanding a detailed compositional analysis of table entries. This dataset uses CC-BY-SA-4.0 license.
- **FeTaQA** features free-form questions derived from tables that call for profound reasoning and comprehension. Predominantly, the questions in FeTaQA arise from non-contiguous segments of the table. The performance is measured by accuracy on a test set of 2,003 samples. This dataset uses CC-BY-SA-4.0 license.
- **TabFact** serves as a benchmark for table-based fact verification, with crowd workers composing statements from Wikipedia tables. For instance, the statement: "The industrial and commercial panel has four more members than the cultural and educational panel." must be validated as "True" or "False" based on the table information. Accuracy is reported on a smaller test set of 2,024 statements across 298 tables. It uses MIT license.
- **SCB** is a spreadsheet manipulation dataset, which contains 28 spreadsheets collected from the Internet. The 221 spreadsheet control tasks within this dataset are generated using GPT-4, including analyzing sales data, calculating financial metrics, and visualizing data with charts. It uses GPL-3.0 license.
- **SheetRM** introduced in Section 2 comprises of 317 real-world spreadsheet tasks. Each task includes multi-category and long-horizon manipulation sequences, along with a specific reasoning

challenge. This dataset comprehensively assesses the reasoning and manipulation capabilities of LLM-based agents. It also supports finer-grained and more flexible automatic evaluation. For more details of our proposed SheetRM benchmark, please refer to Appendix A. Our SheetRM dataset follows the CC-BY-SA-4.0 license.

D Implementation Details

Baselines. As for table reasoning tasks, we run Binder and DATER using the official implementations. The only difference is that we revise the code to use publicly available *gpt-3.5-turbo-16k-0613* as the LLM backbone instead of Codex due to its inaccessibility. We also run StructGPT on TabFact small-test set and FeTaQA using its open-sourced code with the same LLM backbone. On the proposed SheetRM, we have improved SheetCopilot based on the simplified open-source version² with error feedback functionality for fair comparison. For the VBA method, we adjust the prompt of SheetAgent to generate *pywin32* code and remove the Retriever module due to code repository mismatch. FormaT5 and SpreadsheetCoder are implemented using the official open-sourced code. For the rest baselines, we report the performance obtained from papers.

LLM Backbones for SheetAgent. In the main experiments, we select various LLMs as the backbones for our proposed SheetAgent. As for proprietary LLMs, we choose GPT-3.5, GPT-4³, and Claude 3 (claude-3-sonnet-20240229⁴). In terms of open-source LLMs, we adopt Qwen-1.5 (14b-chat⁵) and Llama 3 (8b-instruct⁶). Multiple versions of GPTs are involved for alignment with other baselines. Specifically, for SCB, WikiTableQuestions, FeTaQA, and TabFact, we use *gpt-3.5-turbo-16k-0613*. For our SheetRM, we employ *gpt-3.5-turbo-1106* and *gpt-4-turbo-0409*. For the embedding model in the Retriever, we adopt *text-embedding-ada-002*⁷.

Choice of In-context Examples. For the SCB dataset, we align with SheetCopilot by using only one in-context example. For other datasets, we utilized two in-context examples each. Specifically, for SCB, we selected the same task as SheetCopilot. We initially had VBA and SheetAgent generate a trajectory for the task under a zero-shot setting using GPT-4, then made appropriate modifications to ensure correctness. The modified trajectory was ultimately used as the in-context example. For SheetRM benchmark, we constructed two additional tasks not present in the dataset and employed VBA, SheetAgent and SheetCopilot to generate trajectories for these tasks, in the same manner, to serve as in-context examples. For other datasets, including WTQ, FeTaQA, and TabFact, where our experiments were conducted on the test sets, we chose two tasks from their respective training sets as examples. It is worth noting that our SheetAgent only uses 2 in-context examples while Binder uses 14. SheetAgent still achieves superior performance.

Computing Power. All the results in our experiments are obtained by running the code on a server equipped with an Intel(R) Xeon(R) Gold 6338 CPU @ 2.00GHz and 2*NVIDIA A800.

²<https://github.com/BraveGroup/SheetCopilot>.

³<https://platform.openai.com/docs/models>

⁴<https://docs.anthropic.com/claude/docs/models-overview>

⁵<https://huggingface.co/Qwen/Qwen-14B-Chat>

⁶<https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>

⁷See <https://platform.openai.com/docs/models/embeddings>

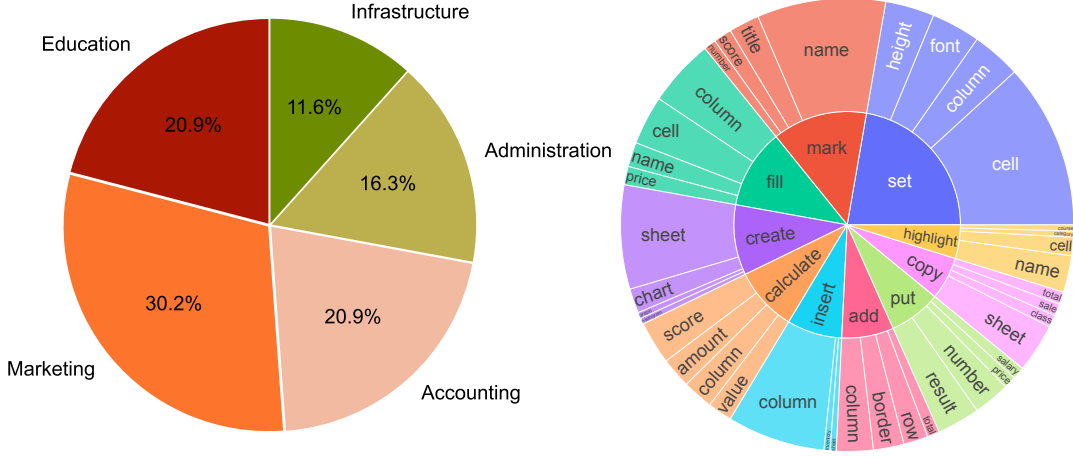


Figure 6: (Left) Distribution of fields to which the spreadsheet files belong. (Right) An illustration of verb-noun phrases in the task instructions. We count the top 10 most frequent root verbs and their associated nouns, ranking the top four for each. These verb-noun combinations showcases the diversity of the generated instructions.

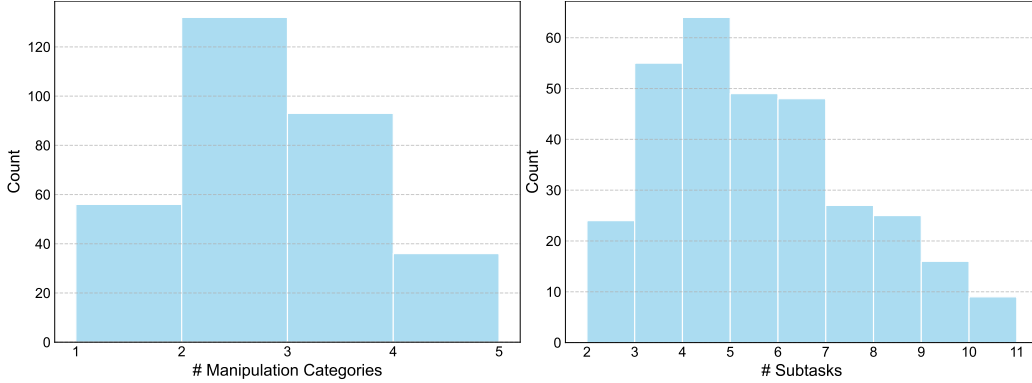


Figure 7: (Left) Distribution of manipulation categories in each task. (Right) Histogram of the task length.

E Additional Experimental Results

E.1 Full Results on Table Reasoning Tasks

Table 9-10 show the full evaluation results on table reasoning tasks.

E.2 Vision-Enabled SheetAgent

We have explored the potential of leveraging GPT-4V(ision)’s visual capabilities by substituting spreadsheet snapshots for the text-modal sheet state in the observation. Given the cost of GPT-4V and the challenges in automatic snapshot capture of spreadsheets, we test this approach with 10 representative tasks from the SheetRM dataset. We have ensured these tasks span all five manipulation categories defined in SheetRM. As vision-enabled SheetAgent can observe full state of spreadsheets, we remove the Informer module for fair comparison. Results are presented in Table 11. Through this intriguing experiment, we observe that when tasks involved visual elements, such as formatting and chart modification, SheetAgent can better adjust styles based on the visual feedback, improving task completion. However, SheetAgent (GPT-4V) has difficulty processing large-scale spreadsheets and correctly identifying sheet

data due to low image solution, resulting in task failure. We also obtain an interesting finding that there exists an overlap between the visual aspect of GPT-4V and the Informer module. While GPT-4V allows SheetAgent to perceive multimodal content within spreadsheets (charts, pivot tables, frozen panes, etc.), it faces challenges capturing accurate information in larger-scale tables compared with the Informer. We leave this for our future work.

E.3 Ablation Study on Table Representation

Tabular data is a kind of information-dense structured data, it is crucial to design reliable representations to enable LLMs clearly recognize the attribute relationships. To investigate what representation can better help LLMs to reason over tables. We ablate 4 prevalent table representations: JSON, DFLoader, Markdown, and HTML for SheetAgent on WikiTableQuestions and SheetRM. We provide an illustration of these representations, as shown in Figure 9. Notably, DFLoader is represented by the corresponding Python code snippet that uses the *pandas* DataFrame API to define the table. The results shown in Table 5 reveal that JSON outperform other

Table 7: A short natural language description of the spreadsheet files we collect in SheetRM dataset.

Spreadsheet File	Description
BookSales	This workbook presents data related to book sales.
StudentsGrade	This workbook is about organizing and analyzing student transcripts for first-grade students.
ABProductSales	This workbook presents data related to product A and B.
Reimbursement	The workbook shows the company’s travel expense reimbursement status for the year 2013.
ElectronicsSales	The workbook is about conducting statistical analysis of the company’s sales.
PayrollSummary	The workbook is the March 2014 employee salary sheet.
TeachingFees	This workbook shows the teaching situation and instructor hourly fees for the courses in the Teaching Research Office in the year 2012.
Deposit	The workbook is a bank deposit journal.
ComputerBookSales	The workbook depicts the sales figures for computer-related books in December 2012.
ScienceMajorGrade	The workbook shows the final exam grades for the Information and Science major.
PersonnalInformation	This workbook is the personnel file information of company employees.
ComputerBookSales2	This workbook represents the sales statistics of computer-related books.
AppliancesSales	This workbook shows the sales statistics of various household appliances.
DepartmentSales	This workbook documents the sales performance of company’s products in the first half of the year.
QuartersSales	This workbook summarizes the sales performance for the first two quarters.
FinalGrade	This workbook provides a detailed analysis of students’ final grades.
ParkingFees	This workbook keeps track of parking fees and the associated rates.
LivingCosts	This workbook displays an individual’s monthly expense report.
StudentsGrade2	This workbook displays the grades for each subject in the class.
LawMajorGrade	This workbook presents the final grade analysis of law students from the 2012 cohort.
YearsSales	This workbook documents the sales statistics of company products in 2012 and 2013.
YearEndSalary	This workbook provides the year-end salary details of employed staff members.
AirQuality	This workbook illustrates the air quality data for major cities in China.
SalesAndPurchase	This workbook is a record of this year’s sales and purchase data.
PersonnelChange	This workbook contains the personal details of company employees for the year 2019, including their entry and departure information.
ProductLaunchPlan	This workbook outlines the product launch timeline, key milestones, and marketing strategies.
StudentAttendance	This workbook tracks the attendance records of students across various grades.
QuarterlyEarnings	This workbook presents the company’s earnings and financial reports for each quarter of 2020.
OfficeInventory	This workbook lists office supplies, including current stock levels and reorder statuses.
RoadMaintenanceLog	This workbook logs the maintenance schedule and costs associated with road repairs in the city.
CustomerSurvey	This workbook compiles customer feedback from recent marketing campaigns and product surveys.
TeacherPerformance	This workbook evaluates teacher performance based on student feedback and exam results.
BudgetForecast	This workbook forecasts the company’s budget allocations for the next fiscal year.
HRLeaveTracker	This workbook tracks employee leave, including vacation days and sick leave balances.
BridgeInspection	This workbook contains the results of bridge safety inspections conducted in 2021.
CampaignROI	This workbook analyzes the return on investment (ROI) of various marketing campaigns.
CourseEnrollments	This workbook tracks student enrollment numbers for various courses during the academic year.
TaxFilingSummary	This workbook summarizes the company’s tax filings for the past three years.
MeetingMinutes	This workbook records the minutes and action items from weekly department meetings.
PowerGridStatus	This workbook monitors the status of the city’s power grid, including outages and repairs.
AdBudgetAllocation	This workbook details the allocation of the advertising budget across different channels.

formats. HTML format achieves a suboptimal result on WTQ, but ranks lowest on SheetRM. Its open-and-close structure helps LLMs understand better, but the verbosity risks exceeding token limits, thus hindering efficiency. We also observe that DFLoader format achieve commendable results, possibly due to its code structure, which might be easier for LLMs to comprehend. Overall, JSON is

a preferable choice for both reasoning intensive tasks, like WTQ, and long-horizon tasks with fewer reasoning elements, such as SheetRM.

Table 8: Description of each fine-grained operation involved in SheetRM dataset.

Manipulation Category	Operation	Description
Value Processing	Calculate	Calculations and statistics.
	Insert	Insert rows or columns.
	Delete	Delete cells, rows or columns.
	Auto Fill	Fill according to the control relationship.
	Sort	Sort rows or columns in ascending or descending order.
	Copy & Paste	Copy and paste cell values.
	Replace	Replace the values of a cell at a specified location.
	Hyperlink	Set up hyperlinks.
	Distinction	Remove duplicates.
	Filter	Filter specified cells according to certain conditions.
Worksheet Management	Create Worksheet	Create a new worksheet.
	Delete Worksheet	Delete the specified worksheet.
	Rename Worksheet	Rename the specified worksheet.
	Label Color	Modify the color of worksheet name labels.
	Page Size	Modify page size.
Format Adjustment	Orientation	Set the page orientation.
	Font Name	Set the font category.
	Font Color	Set the font color.
	Font Size	Set the font size.
	Bold & Italic	Set the font to be bold or slanted.
	Underline	Underline cell contents.
	Merge & Unmerge	Merge or split cells.
	Alignment	Align cells horizontally or vertically.
	Row Height & Column Width	Set cell row height or column width.
	Background Fill	Set cell background fill color.
Chart Design	Numeric Format	Set cell number formatting.
	Chart Type	Set the Chart Type.
	Chart Data Source	Set the data source for the chart.
	Chart Caption	Set the title of the chart.
	Chart Legend	Set the Chart Legend.
	Chart Position	Specify where to place the chart.
	Chart Axis	Set the axes of a chart.
	Data Label	Set data labels for charts.
Content Summary	Trendline	Add a trendline to the chart.
	Pivot Creation	Create pivot table.
	Summary Function	Set statistical functions of the pivot.

E.4 Ablation Study on LLM Temperature

We conduct evaluations of our method using the proposed SheetRM dataset under varying conditions by adjusting the temperature settings to investigate the impact of temperature on the performance of LLMs. For these experiments, gpt-3.5-turbo-1106 is selected as the LLM backbone. Our findings reveal that our method, SheetAgent, achieves its best performance at a temperature of 0.0, with minor performance fluctuations observed at a temperature of 0.2. However, a noticeable decline in performance across all metrics

occurs when the temperature is increased to 0.4. This trend suggests that higher temperature settings lead to more unpredictable outcomes from SheetAgent, reflecting a decrease in the stability and reliability of the solutions it generates.

E.5 Performance-Cost Analysis

We have conducted extra experiments to calculate the token and time consumption of our method on SheetRM. We use GPT-3.5 as the LLM backbone. We compare our proposed SheetAgent with

Table 9: Results of different methods on WTQ test set and TabFact small-test set. We report the accuracy metric. Best results are bolded and second-best results are underlined.

Method	WTQ	TabFact
Fine-tuning based LLMs		
TAPAS [15]	48.8	83.9
TAPEX [22]	57.5	84.2
UnifiedSKG [39]	49.3	85.4
OmniTab [17]	<u>62.8</u>	82.8
Prompting based LLMs		
GPT-3 CoT [4]	45.7	76.0
Binder [7]	59.9	82.9
DATER [40]	61.6	80.7
StructGPT [16]	52.2	81.2
SheetAgent (Ours)	64.4	<u>84.8</u>

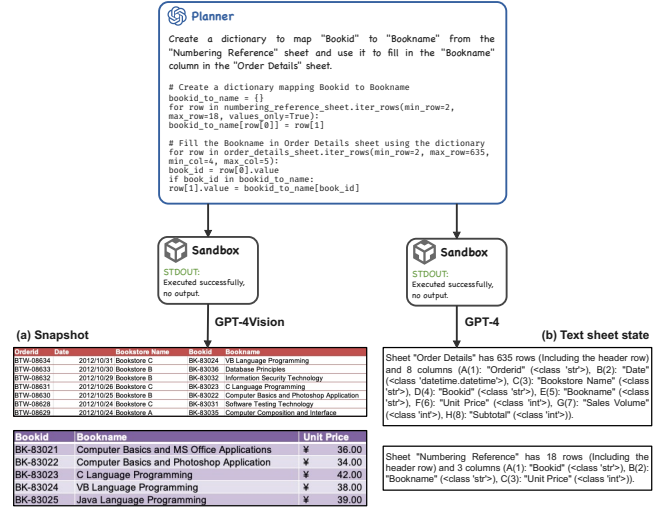
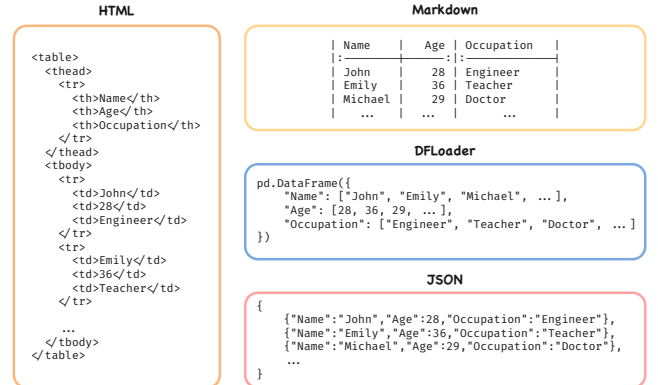
Table 10: Results of different methods on FeTaQA test set. Best results are bolded and suboptimal results are underlined.

Method	sacreBLEU
Fine-tuning based LLMs	
T5-small [23]	21.6
T5-base [23]	28.1
T5-large [23]	30.5
TAPEX [22]	34.7
UnifiedSKG [39]	33.4
PeaQA [25]	33.5
OmniTab [17]	<u>34.9</u>
Prompting based LLMs	
GPT-3 CoT [4]	27.0
Binder [7]	31.6
DATER [40]	30.9
StructGPT [16]	32.5
SheetAgent (Ours)	36.7

Table 11: Performance comparison between SheetAgent (GPT-4) and SheetAgent (GPT-4V) on 10 representative tasks from SheetRM. Vision-enabled SheetAgent removes the Informer module.

Method	Pass@1 ↑	SubPass@1 ↑
SheetAgent (GPT-4V)	40.0	66.5
SheetAgent (GPT-4)	50.0	74.1

SheetCopilot. It is noteworthy that since SheetCopilot is insufficiently capable of fulfilling a complete task from SheetRM, we select 20 subtasks for which both can generate successful trajectories and calculate metrics based on these. The results are presented in Table 13. On average, our approach consumes fewer tokens compared to SheetCopilot, primarily because SheetCopilot often makes errors,

**Figure 8: An illustration depicting the differences in sheet state between GPT-4V and GPT-4. For brevity, the Informer and Retriever modules are excluded. The snapshots (namely the visual representation of sheet state) are partial due to the limitation of spreadsheet scale.****Figure 9: An illustration of four different table representations we use in our experiments.****Table 12: Ablation study on the temperature of LLM.**

Temperature	Exec@1 ↑	Pass@1 ↑	SubPass@1 ↑
0.0	92.4	31.2	69.8
0.2	93.5	30.3	68.1
0.4	90.2	28.7	66.2

which leads to reflection. However, SheetAgent involves querying multiple LLMs and a vector database, which places us at a disadvantage in terms of time efficiency.

As for cost, we have calculated the cost of successful trajectories by our SheetAgent (GPT-3.5) on SheetRM. The average cost to

Table 13: Token and time consumption comparison. Consumption of tokens is calculated by stage.

Method	Avg. # Tokens	Avg. Time (s)
SheetAgent (GPT-3.5)	System Prompt: 324 + Few-shot Demonstrations: 2013 + Planner: 589.7 + Informer: 513.9 + Retriever: 625.3 = 4065.9	6.9
SheetCopilot (GPT-3.5)	System Prompt: 1895 + Few-shot Demonstrations: 1592 + Plan: 628.5 = 4115.5	5.8

</

Figure 10: A comparison between SheetAgent and SheetCopilot on a spreadsheet task with reasoning challenges. SheetCopilot generates a rigid solution that fails to fulfill the instruction. SheetAgent identifies the task intention and gives a correct solution.

finish a complete task is \$0.0049. Notably, excellent results of SheetAgent shown in Table 2 can be achieved even with relatively cheap backbone GPT-3.5 and Claude 3 Sonnet, which is a trade-off between cost and performance. We believe the superior performance of SheetAgent compared to other methods justify this resource use.

F An Illustrative Case Between SheetAgent and SheetCopilot

Figure 10 presents a case with reasoning challenge. For SheetAgent (GPT-3.5), the Informer accurately selects the key evidence related to the task instruction. The Planner correctly fulfill the task based on the evidence. In contrast, SheetCopilot (GPT-4) merely offers a rigid solution that fails to complete the task despite its successful execution.

G Failure Cases Analysis

To gain a clearer understanding of the differences between the LLM backbones compared in Sections Section 4.2 and Section 4.3, we conduct a detailed error analysis to determine the causes and locations of failures. We classify the reasons for failure as follows:

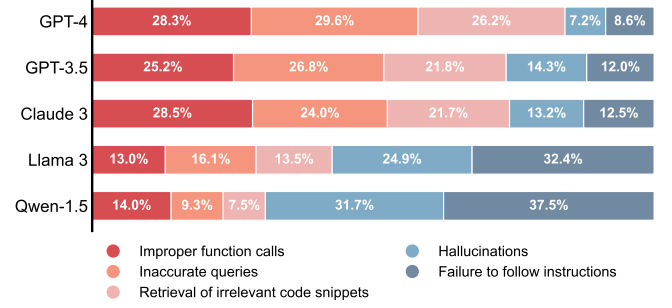


Figure 11: Distributions of different error cases for various LLM backbones on the SheetRM dataset.

- **Improper function calls:** The Planner inaccurately invokes function interfaces for certain operations. For example, it uses `chart.set_title("Chart Title")` instead of `chart.title = "Chart Title"`, resulting in an `AttributeError`. Additionally, it performs `worksheet.cell_range()` (a method deprecated in newer versions of `openpyxl`) instead of `worksheet.iter_rows()` or `worksheet.iter_cols()`.
- **Inaccurate queries:** The Informer generates imprecise SQL statements leading to incorrect or incomplete information being input. For example, it attempts to query books related to office software but produces a statement like `SELECT * FROM w WHERE BookName LIKE '%Excel%' OR BookName LIKE '%PowerPoint%'` while ignoring Word.
- **Retrieval of irrelevant code snippets:** The Retriever sources irrelevant code fragments, which impedes the Planner’s correction process. This happens due to similarities in code segments within the code repository, resulting in the retrieval of irrelevant code.
- **Hallucinations:** It performs operations on rows and columns that are out of scope, ignoring the actual data, or creates data that does not exist.
- **Failure to follow instructions:** It terminates tasks prematurely or fails to comply with the given instructions. For example, it only completes a part of subtasks from a long-horizon task or highlights entries in colors not specified by the task.

We present the proportions of different failure cases for various LLM backbones on our SheetRM in Figure 11. Combining the results in Figure 4, we can observe that: (i) LLMs with poorer performance on the benchmark (e.g., llama3-8b-instruct and qwen-14b-chat) have a significantly higher proportion of errors related to “hallucinations” and “failure to follow instructions”. In contrast, well-tuned LLMs with extensive parameters, such as GPT-3.5, GPT-4, and Claude 3 Sonnet, have their errors predominantly concentrated in “improper function calls”, “inaccurate queries”, and “retrieval of irrelevant code snippets”. This indicates that advanced LLMs perform better on complex tasks, whereas smaller open-sourced LLMs struggle significantly with understanding and executing instructions. (ii) Among proprietary LLMs, GPT-3.5 and GPT-4 exhibit similar error distributions, with high proportions of errors in “inaccurate queries” and “improper function calls” (29.6% and 28.3% for GPT-4, 26.8% and 25.2% for GPT-3.5, respectively). In contrast, Claude 3 Sonnet shows a different pattern, with a similar proportion of “inaccurate

queries” (24.0%) but a relatively higher proportion of “improper function calls” (28.5%). This may reflect that GPTs are adept at generating proficient Python code, while Claude can better understand complicated instructions and translate them into accurate SQLs. (iii) Smaller open-source LLMs, such as llama3-8b-instruct and qwen-14b-chat, display similar error patterns, primarily in “hallucinations” and “failure to follow instructions”. Llama3-8b-instruct possesses a “failure to follow instructions” error rate of 32.4%, whereas qwen-14b-chat has a significantly higher rate of 37.5%. This phenomenon may be attributed to their training corpus and model scale.

We further perform a deep analysis of specific failure cases across different LLM backbones, which reveals distinct patterns and challenges. GPT-4 and GPT-3.5 are prone to make errors in generating correct SQLs. After inspecting the specific bad cases, we find that in most cases, they understand the task instruction but generate SQLs that semantically fail to fulfill the task requirements. In other cases, they generate syntactically incorrect SQL statements that cause execution to fail. Differently, Claude 3 Sonnet owns a highest rate of improper function calls but fewer errors of inaccurate queries. It usually calls a function that does not exist or is deprecated, or misunderstands the function usage. For instance, it uses *openpyxl*’s `iter_rows()` function to iterate through the spreadsheet. The exact code it produces is for `row in ws.iter_rows(min_row=1, max_row=10, max_col="E"):`, where `max_col` should be an integer instead of a string. Llama3-8b-instruct and qwen-14b-chat share the highest proportions of instruction-following failures and hallucinations, suggesting difficulties in maintaining task context and adhering to long-horizon instructions. We note that there are a large number of incomplete solutions in the llama3-8b-instruct failure case, due in large part to its limited context length of 8K. For qwen-14b-chat, we observe that it can hardly follow the complicated and long-horizon task instructions, and tends to generate irrelevant contents. We assume this may have something to do with its training strategy and corpus.

We have further proposed potential strategies to overcome the proposed failure cases, which may provide insights for future research in this community: **Regarding addressing improper function calls**, we found conflicts between the LLM’s training corpus on *openpyxl* versions and current versions. Enhancing understandings of library functions through fine-tuning or tool augmentation might mitigate this. **For inaccurate queries**, improving model training with diverse SQL examples through fine-tuning and incorporating a validation layer to check queries against database schemas could enhance accuracy. **To combat irrelevant code snippet retrieval**, refining the code repository with detailed descriptions of each example’s functionality and intended task scenarios could improve retrieval accuracy. **For hallucinations and failure to follow instructions**, we attribute these to the model’s inherent limitations, noticing a significant increase in these issues on weaker LLM backbones like llama3-8b-instruct and qwen-14b-chat. Switching to a more robust LLM might alleviate these problems. Explicitly managing task progress (e.g., adding a task decomposition module for procedural execution) or incorporating an LLM-driven Critic module (for sanity check on generated solutions) could also partially address these issues.

H Prompts

We provide the prompts for Planner and Informer, as shown in Figure 12 and Figure 13. Additional prompts for subtask generation can be found in our project website.

I Limitations and Potential Social Impact

We list the limitations of our proposed SheetAgent as follows:

- **Library limitations:** SheetAgent automates spreadsheet tasks through Python code generation, utilizing libraries like *openpyxl* and *pandas*. Although this has covered a wide range of operations, it is still missing some customizable functionality at the software level. For instance, complex spreadsheet manipulations that involve advanced Excel features such as pivot tables, macros, or specific formatting options are not fully supported. Enhancements in library capabilities or integration with additional tools could address these gaps.
- **High token usage:** Like existing research to automate spreadsheet manipulation [19], SheetAgent inevitably faces higher token usage for long-horizon tasks. This can lead to increased computational costs and slower processing times. Future work will focus on optimizing task descriptions through more efficient prompting techniques or manual refinement to reduce token consumption and improve overall efficiency.

The implementation of SheetAgent has the potential to bring about several positive social impacts. By automating repetitive and time-consuming spreadsheet tasks, SheetAgent can significantly enhance productivity and efficiency in various industries. This can free up human resources for more strategic and creative work, ultimately leading to better utilization of talent and skills. Additionally, SheetAgent can democratize access to advanced data analysis and processing, making these capabilities available to a broader audience, including individuals with limited technical expertise. This democratization can empower more people to leverage data for informed decision-making and innovation.

However, the introduction of SheetAgent might pose negative social impacts. As with any automation technology, there is a risk of job displacement for roles traditionally centered around manual spreadsheet manipulation. This could lead to economic and social challenges for affected individuals. Moreover, the reliance on computational resources for running SheetAgent, especially for large-scale or long-horizon tasks, could contribute to environmental concerns such as increased energy consumption. Addressing these issues requires proactive measures, including reskilling and upskilling programs to help displaced workers transition to new roles and optimizing the efficiency of SheetAgent to minimize its environmental footprint. Ethical considerations must also be prioritized to ensure transparency, fairness, and the safeguarding of user data privacy and security.



System prompt

Role

You are a spreadsheet agent and a python expert who can find proper functions to solve complicated spreadsheet-related tasks based on language instructions.

Prerequisites

1. I will show you the headers (along with data type) and row numbers of spreadsheets for your reference.
2. Your partner, "Informer," aids in task completion by providing sheet content represented in {table_representation}, known as "potentially helpful information". This information might be truncated due to token limits, so it's essential to deduce the complete information from what is provided.
3. Please provide step-by-step solutions without explanation.
4. You can use any python library, but when it comes to manipulating spreadsheets, you should primarily use the openpyxl and pandas library, which has been already imported as `openpyxl` and `pd`.
5. You should only give one python code snippet at a time. Try not to add comments, and if you must, keep them as concise as possible.
6. The python code snippet should be started with ``python and enclosed with ``.
7. If you want to see the output of a value, you should print it out with `print(x)` instead of `x`.

Response Format Guidance

1. If you think a python code snippet is needed, write using the following output format:

Think: (what you need to solve now and how to solve)

Action: Python

Action Input: (your python code snippet, which should be in accordance with above prerequisites)

2. If you think there is a question to be answered, give your answer using the following format:

Think: (how do you get the answer)

Action: Answer

Action Input: (your answer)

2. If you think task instruction is accomplished, finish with the following format:

Finish: Done!

In context example

...

Instruction

Now it's your turn. This Workbook presents data related to book sales. The workbook is already loaded as `workbook` using openpyxl, you only need to load the sheet(s) you want to use manually. Besides, the workbook will be automatically saved, so you don't need to save it manually.

Sheet state: Sheet "Order Details" has 635 rows (Including the header row) and 8 columns (A(1): "Orderid" (<class 'str'>), B(2): "Date" (<class 'datetime.datetime'>), C(3): "Bookstore Name" (<class 'str'>), D(4): "Bookid" (<class 'str'>), E(5): "Bookname" (<class 'NoneType'>), F(6): "Unit Price" (<class 'int'>), G(7): "Sales Volume" (<class 'int'>), H(8): "Subtotal" (<class 'int'>)). Sheet "Numbering Reference" has 18 rows (Including the header row) and 3 columns (A(1): "Bookid" (<class 'str'>), B(2): "Bookname" (<class 'str'>), C(3): "Unit Price" (<class 'int'>)). Sheet "Statistical Report" has 5 rows (Including the header row) and 2 columns (A(1): "Statistical Item" (<class 'str'>), B(2): "Statistical Value" (<class 'int'>)).

Task instruction: Format the "Order Details" worksheet, adjust "Unit Price" and "Subtotal" to accounting with 2 decimal places and CNY currency symbol format. Fill the "Bookname" column based on Bookid. Identify the bookstore with the largest decrease in sales subtotal in 2012 compared to January to October 2011. Then, determine the best-selling book in that year for this bookstore and highlight its row in red in the "Numbering Reference" worksheet.

Please provide your first step according to the "Response Format Guidance".

Planner response

Think: Apply accounting format with 2 decimal places and CNY currency symbol to the "Unit Price" and "Subtotal" columns in the "Order Details" worksheet.

Action: Python

Action Input:

``python

Load the "Order Details" sheet

order_details_sheet = workbook["Order Details"]

Apply accounting format with 2 decimal places and CNY currency symbol to "Unit Price" and "Subtotal" columns

for cell in order_details_sheet["F"][1:] + order_details_sheet["H"][1:]:

cell.number_format = '"¥"#,##0.00_);[Red]('"¥"#,##0.00)'

...

.....

Figure 12: A prompt template for the Planner.



Informer

System prompt

You are a table retrieval expert who team up with a spreadsheet agent named "Planner" to accomplish complicated spreadsheet-related tasks. Your partner excel in manipulating spreadsheets. However, some of manipulations require an understanding of specific content of the spreadsheet. Therefore, your role is to retrieve key information from spreadsheets for your partner's reference.

In context example

...

Instruction

Now it's your turn. To make it easier for you to retrieve, all sheets are stored in a SQLite database.

Table schema of "First semester final grades":

```
CREATE TABLE "First semester final grades" (
  "Student Number" INTEGER,
  "Name" TEXT,
  "Class" REAL,
  ...
)
```

3 example rows:

```
SELECT * FROM "First semester final grades" LIMIT 3;
{
  "1":{"Student Number":120305,"Name":"Alice","Class":null,"Chinese":91.5,"Mathematics":89,"English":94,"Biological":92,"Geography":91,"History":86,"Politics":86,"Total Score ":null,"Average Score":null},
  "2":{"Student Number":120203,"Name":"Charlie","Class":null,"Chinese":93.0,"Mathematics":99,"English":92,"Biological":86,"Geography":86,"History":73,"Politics":92,"Total Score ":null,"Average Score":null},
  ...
}
```

Task instruction: Calculate the total score and average score of each student, and fill in the "Total Score" and "Average Score" columns respectively. Fill in the "Class" column according to the "Student Number". There are three classes: 1, 2, and 3. Fill the cells with scores of no less than 110 points in the three subjects of Chinese, Mathematics, and English in red respectively, and mark the cells with scores above 95 points in the other four subjects in red font.

Previous completed subtasks of the spreadsheet agent (which is presented from your partner's point of view and is more of your partner's thought while finishing the subtasks):

1. Load the "First semester final grades" sheet and calculate the total and average scores for each student.

Given the task instruction and subtasks your partner has completed, predict what your partner will solve next, and determine what content is most needed by your partner. Write a SQLite select statement to achieve this goal. If there is no need to retrieve information from any spreadsheet, please type "pass".

Based on above, responde in the following format:

Think: (how do you think)

Action: (your SQLite statement, e.g., `SELECT \* FROM w WHERE age < 18;`, or "pass")

Informer response

Think: According to the task instruction and completed subtasks, the next step for the Planner is more likely be to fill the "Class" column. The rule for filling "Class" depends on the "Student Number" column, so I should provide the agent with the "Student Number" information.

Action: SELECT "Student Number" FROM "First semester final grades" LIMIT 3;

Figure 13: A prompt template for the Informer.